

AsyncMux: Event-Driven Coordination for Multiplexed Tiered Storage

Cole Harten
Harvard University

Abstract

Modern storage systems increasingly combine heterogeneous devices such as DRAM-backed filesystems, SSDs, and HDDs to balance latency, throughput, and capacity. Multiplexed storage layers expose these tiers through a unified namespace, but make a single logical operation harder to implement: one request may require locating blocks across devices, issuing multiple I/O operations, waiting for all of them to finish, and assembling the final result.

This paper presents *AsyncMux*, a prototype multiplexed storage layer that uses coroutine-based asynchronous coordination to express these multi-tier operations as composable tasks. AsyncMux executes block-level work concurrently without dedicating a blocked thread to each sub-operation, making fanout-style coordination simpler to express and potentially more scalable under load.

We compare AsyncMux against *BlockingMux*, a matched blocking baseline that implements the same storage abstraction, metadata structures, and placement policies. Across sequential, random, mixed, and multi-tier fanout workloads, we evaluate throughput, tail latency, and scalability under increasing concurrency. Our results show that asynchronous coordination is especially effective for read-heavy and fanout workloads, where it improves throughput and scalability, but that these gains come with tradeoffs in tail latency under higher concurrency.

1 Introduction

Modern storage stacks are increasingly heterogeneous. Applications routinely access data spread across multi-

ple tiers, including memory-backed filesystems, SSDs, and lower-cost disk storage. Multiplexed storage systems unify these devices behind a single logical filesystem and allow data to be placed, migrated, and accessed transparently.

The benefit of multiplexing is flexibility; the cost is coordination. A logical request may span blocks on different devices, requiring the system to translate file offsets into physical locations, issue multiple I/O operations, track their completion, and merge the results into one response. Writes are similarly complex: data must be split into blocks, mapped onto tiers, and committed without corrupting metadata or exposing inconsistent state.

A simple example illustrates the challenge. A 64 KB logical read may span sixteen 4 KB blocks, with some blocks on a hot tmpfs tier, others on SSD, and others on HDD. Serving that read requires finding the block locations, dispatching all needed reads, waiting for them as a group, and copying the returned data into a contiguous output buffer. In a blocking design, this kind of fanout often leads to thread orchestration, condition variables, and cross-thread synchronization. In an asynchronous design, the same operation can be written as a collection of tasks awaited together.

This paper asks whether asynchronous coordination can simplify multiplexed storage operations without sacrificing performance. We present *AsyncMux*, a prototype tiered storage layer implemented in C++ using coroutines. AsyncMux models each logical request as a composition of block-level asynchronous tasks. These tasks may run concurrently, suspend while waiting for I/O, and resume when ready, allowing the logical structure of the operation to be expressed directly.

To isolate the effect of the execution model, we also

implement *BlockingMux*, a blocking baseline with the same storage abstraction, metadata store, placement policy, and migration behavior. The two systems differ primarily in how they coordinate multi-block work: *BlockingMux* relies on threads and blocking synchronization, while *AsyncMux* uses coroutine-based task composition over a bounded worker pool.

Our results show a more nuanced picture than “async is better.” *AsyncMux* performs best on read-heavy and fanout workloads, where one logical request naturally decomposes into many independent sub-operations. Under these conditions, it achieves higher throughput and scales better with concurrency. However, it does not dominate the blocking design on all workloads, and it can incur worse tail latency when many tasks compete for shared runtime resources.

The contributions of this paper are:

- The design and implementation of *AsyncMux*, a coroutine-based multiplexed storage layer that coordinates block-level I/O across multiple storage tiers using asynchronous task composition.
- A matched blocking baseline, *BlockingMux*, that shares the same storage abstraction, metadata structures, and placement policies, enabling the execution model to be isolated experimentally.
- An evaluation showing that asynchronous coordination improves throughput and scalability on read-heavy and fanout workloads, while also exposing tradeoffs in tail latency and coordination-heavy cases.

2 Related Work

Tiered File Systems. Tiered storage systems expose heterogeneous devices through a unified filesystem and dynamically place or migrate data across tiers. Systems such as *Strata* [2] and *Ziggurat* [5] directly manage multiple devices within a monolithic file-system design. These systems primarily study how data should be placed and moved across the hierarchy.

Mux [4] takes a different approach by composing existing device-specific file systems beneath a multiplexing layer. This separation improves extensibility and allows tiering to reuse optimized underlying file systems. *AsyncMux* builds on this direction, but focuses on a different question: once data has been distributed

across tiers, how should multi-tier operations be coordinated?

Specialized File Systems. Recent systems have explored file systems specialized for emerging storage technologies, such as persistent memory, including *NOVA* [3] and *SplitFS* [1]. These systems improve performance by matching file system structure to device characteristics. *AsyncMux* does not introduce a new device-specific file system; instead, it treats underlying tiers as black boxes and studies the coordination layer above them.

Union and Meta File Systems. Union-style systems such as *OverlayFS* and *UnionFS* combine multiple file systems into a single namespace, but generally operate at file granularity. In contrast, tiered systems such as *Mux* and *AsyncMux* distribute data at block granularity. As a result, a single user-visible operation may require multiple reads or writes across different tiers, making intra-request coordination central to performance.

Concurrency in Storage Systems. Many storage systems rely on blocking, thread-based coordination for parallel I/O. *Mux* discusses concurrency in the context of migration and optimistic concurrency control, but not the structure of ordinary multi-tier request coordination. *AsyncMux* instead examines the execution model itself. The central question is not only where blocks live, but how a system should orchestrate the many sub-operations needed to access them efficiently.

3 Design

This section describes *AsyncMux* (*AMux*) and *BlockingMux* (*BMux*), two implementations of a multiplexed tiered storage layer that expose a unified namespace over multiple storage tiers. The systems share the same storage model, metadata structures, and placement policy, and differ primarily in how they coordinate multi-tier operations.

3.1 Overview

AMux and *BMux* present one logical filesystem backed by multiple tiers, such as *tmpfs*-, *SSD*-, and *HDD*-backed file systems. Files are divided into fixed-size blocks, and

each block is placed on a tier according to a policy. A single logical read or write may therefore span multiple blocks on different devices.

Both systems implement four common responsibilities:

- **Block-level placement:** splitting logical requests into per-block operations and choosing a destination tier for each block.
- **Metadata management:** recording how logical file offsets map to physical storage tiers.
- **Cross-tier coordination:** issuing block-level I/O to multiple tiers and reassembling a logical response.
- **Migration:** moving blocks across tiers to implement promotion and placement policy changes.

The key difference is how these block-level activities are coordinated once a request has been decomposed.

3.2 Design Principle

The central design principle of AsyncMux is to treat each logical operation as a composition of block-level tasks. Reads, writes, and migrations are first translated into per-block work using metadata and placement policy. The system must then coordinate these subtasks and present the result as one logical operation.

BlockingMux handles this coordination with threads and blocking synchronization: worker threads perform per-block I/O, and the caller waits until all work has completed. AsyncMux instead expresses per-block work as asynchronous tasks that can be launched together and awaited collectively using a `when_all`-style primitive. This separates the logical structure of the operation from explicit thread management.

This distinction matters most for fanout-style operations. When many blocks must be fetched from different tiers, the dominant implementation question is how naturally the system can describe and manage the required parallelism.

3.3 Storage Model

Both systems use a block-based abstraction. Files are logically divided into 4 KB blocks, and each block is associated with a storage tier.

Underlying tiers store data using sparse files so that logical offsets are preserved across devices. This avoids an additional address translation step and allows blocks from one logical file to reside on different tiers while still occupying the same offset within each backing file.

3.4 Metadata Management

A shared `MetadataStore` maintains the mapping from logical file state to physical block placement. It stores:

- A per-file extent list, where each extent describes a contiguous byte range stored on one tier.
- A global block index that maps block identifiers to file paths and logical offsets.

When a write overlaps existing extents, the affected extents are split into non-overlapping fragments and replaced as needed. This guarantees that the metadata representation remains consistent and free of overlapping mappings.

The metadata layer is protected by a read-write lock, allowing concurrent lookups while serializing structural updates.

3.5 Placement and Request Splitting

Writes are first divided into block-aligned chunks. Each chunk is then assigned to a tier according to the placement policy. The corresponding block-level writes are issued, and metadata is updated to record the resulting locations.

Reads follow the inverse path: the system consults metadata to identify the blocks and tiers needed to serve a request, then schedules one block-level read per relevant extent.

3.6 Coordination Model

BlockingMux (BMux). BMux coordinates a multi-block request using blocking worker threads. Each block-level read or write is assigned to a worker that performs blocking I/O using calls such as `pread` and `pwrite`. A shared completion counter and condition variable are used to detect when all block-level operations have finished.

This approach is straightforward, but it requires explicit synchronization and dedicates a blocked thread to each in-flight unit of work.

AsyncMux (AMux). AMux coordinates the same block-level work using asynchronous tasks. Each sub-operation is represented as a coroutine task, and groups of tasks are composed with a `when_all`-style primitive. The calling coroutine suspends until the full group completes, then resumes to assemble the result.

In the current prototype, tasks are executed on a shared worker pool. Individual storage operations still ultimately invoke blocking system calls, but the coroutine abstraction allows many logical requests to be multiplexed over a bounded set of threads and avoids exposing thread management in the control flow of the storage layer itself.

3.7 Result Assembly

After block-level reads complete, both systems reconstruct the logical result by copying returned data into the appropriate offsets of a contiguous output buffer. This step hides the physical distribution of data and preserves the appearance of a conventional file interface.

3.8 Migration

Both systems support explicit and background migration of blocks between tiers. Migration proceeds in three steps:

1. Read the block from the source tier.
2. Write the block to the destination tier.
3. Update metadata to record the new location.

To guard against races with concurrent writes, migration uses a simple version-based check. The system records the file version before moving data and verifies that the version has not changed before committing the metadata update. If the version changes, the migration is retried.

3.9 Background Promotion

To support dynamic tiering, both systems include a background mechanism that promotes frequently accessed blocks to a designated hot tier. Reads from non-hot tiers enqueue promotion requests, and a background worker processes those requests without blocking the foreground read.

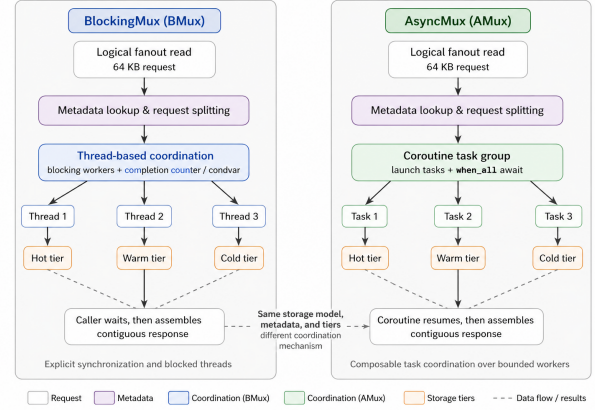


Figure 1: Coordination of a fanout read in BlockingMux (BMux) and AsyncMux (AMux). Both systems perform the same high-level steps—metadata lookup, request splitting, per-block I/O across tiers, and result assembly—but differ in how block-level work is coordinated. BMux uses blocking worker threads and explicit completion synchronization, while AMux expresses the same structure as a group of coroutine tasks awaited together.

4 Evaluation

We evaluate AsyncMux against BlockingMux to understand how the execution model affects throughput, scalability, and tail latency. Because the two systems share the same storage abstraction, metadata structures, and placement policy, the comparison isolates coordination strategy as the main experimental variable.

Our evaluation is organized around three questions:

- How do AMux and BMux compare across representative workloads?
- Does asynchronous coordination improve scalability under concurrency?
- What tradeoffs appear in tail latency?

4.1 Experimental Setup

All experiments were run on a single Linux machine, and three storage tiers: a tmpfs hot tier, an SSD-backed warm tier, and an HDD-backed cold tier. Unless otherwise noted, files are divided into 4 KB blocks and both systems use the same metadata store, placement policy, and promotion mechanism.

The current AsyncMux prototype uses a shared worker pool of 8 threads. BlockingMux uses 8 for the corresponding blocking implementation. Benchmarks report end-to-end performance at the logical operation level rather than the individual block-operation level.

We evaluate seven workloads:

- **Sequential write:** contiguous writes to a logical file.
- **Sequential read:** contiguous reads from a logical file.
- **Random read:** uniformly random reads across a file.
- **Random write:** uniformly random writes across a file.
- **Mixed 80/20:** a workload with 80% reads and 20% writes.
- **Fanout read:** logical reads that span multiple tiers.
- **Multitier RW:** a coordination-heavy read/write workload that exercises placement and cross-tier activity together.

These results should be interpreted with two limitations in mind. First, the current AsyncMux runtime is intentionally simple, so measured behavior reflects both the coroutine abstraction and the characteristics of this worker-pool scheduler. Second, the evaluation is limited to one machine and one placement policy, leaving broader policy interactions and distributed settings to future work.

4.2 Performance Across Workloads

Figure 2 shows throughput across workloads. The clearest advantage for AMux appears on read-heavy workloads, including sequential read, random read, and fanout read. These workloads are a natural fit for block-level asynchronous coordination because one logical request can be decomposed into largely independent sub-operations that can be launched together and awaited as a group.

The picture is more mixed on workloads that combine reads, writes, and cross-tier coordination more tightly. AMux remains competitive on sequential and

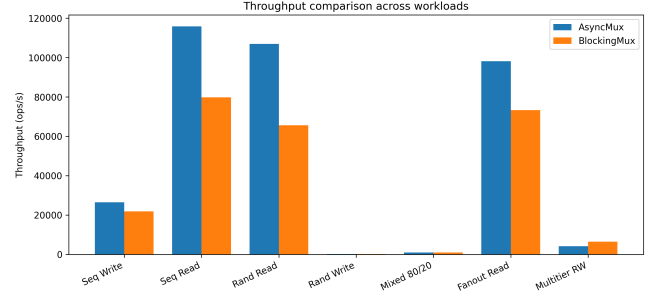


Figure 2: Throughput across workloads. AsyncMux performs best on read-heavy workloads, especially sequential, random, and fanout reads, while BlockingMux remains competitive on coordination-heavy multitier read/write behavior.

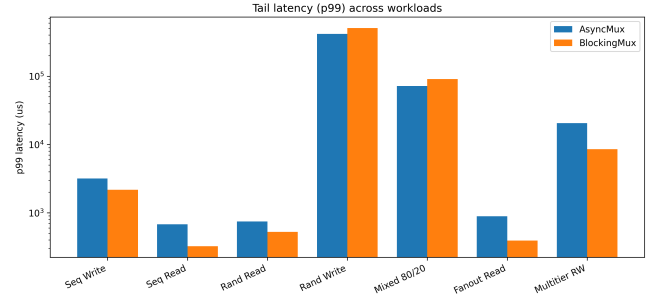


Figure 3: Tail latency (p99) across workloads. AsyncMux improves p99 in some write-oriented cases, but BlockingMux achieves lower tail latency in several read-heavy and multitier workloads.

random writes, but BMux performs better on the multitier read/write case. This suggests that asynchronous composition helps most when independent I/O can be exposed cleanly, and less when requests interact more heavily through shared metadata or competing reads and writes.

Figure 3 presents p99 latency. AMux achieves lower tail latency on some write-oriented workloads, including random write and the mixed 80/20 case. However, BMux produces lower p99 latency on several read-heavy and coordination-heavy workloads, indicating that AMux’s gains in throughput do not always translate into more predictable completion times.

4.3 Scalability on Fanout Reads

Fanout reads are the workload most closely aligned with the AsyncMux design. Serving one logical read may

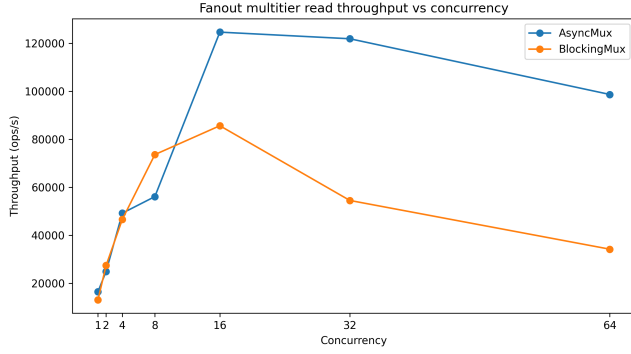


Figure 4: Fanout read throughput versus concurrency. AsyncMux scales more effectively than BlockingMux at moderate and high concurrency, indicating that asynchronous coordination is especially beneficial for fanout-heavy reads.

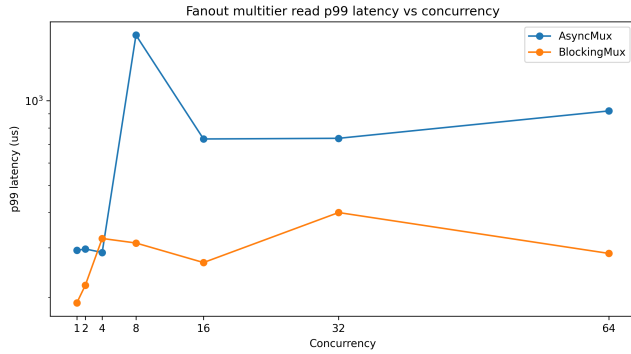


Figure 5: Fanout read p99 latency versus concurrency. Although AsyncMux achieves higher throughput at scale, its tail latency grows more sharply under higher concurrency.

require dispatching requests to multiple tiers and waiting for them all before producing a response.

Figure 4 shows throughput as concurrency increases. At low concurrency, the two systems are broadly comparable. As concurrency rises, however, AMux scales more effectively and achieves substantially higher throughput than BMux. This is consistent with the idea that coroutine-based coordination reduces the overhead of expressing and managing many concurrent sub-operations for one logical request.

Figure 5 shows the corresponding p99 latency. Here the tradeoff becomes clear: while AMux sustains higher throughput, its tail latency increases more sharply at higher concurrency. The most plausible explanation is that tasks contend for shared runtime resources, lead-

ing to queueing and scheduling delay even though the program no longer blocks one dedicated thread per sub-operation.

4.4 Takeaway

Overall, the evaluation shows that asynchronous coordination is not uniformly better, but that it is especially effective for workloads dominated by read-heavy fanout across tiers. In those cases, AsyncMux improves throughput and scales more gracefully with concurrency. The main cost is higher tail latency under heavier load, suggesting that runtime policy is as important as the programming model itself.

5 Discussion

The evaluation points to a central lesson: in a multiplexed storage layer, the execution model matters most when one logical request naturally expands into many independent I/O operations. AsyncMux benefits from this structure because it lets the implementation express those operations directly as a task group rather than as a collection of threads and explicit synchronization points.

Where AsyncMux Helps Most. AsyncMux is most effective on read-heavy and fanout workloads. These workloads contain abundant parallelism within one request, and they spend relatively little time contending on shared metadata compared with issuing and waiting on many block-level reads. In that setting, the coroutine-based approach reduces coordination overhead and makes higher concurrency easier to sustain.

Why BlockingMux Remains Competitive. Blocking coordination remains competitive on workloads with tighter coupling between sub-operations, especially multitier read/write activity. In these cases, the system spends more time in metadata updates, serialization points, or interactions between reads and writes. That reduces the benefit of exposing many independent tasks, and makes the runtime overhead of asynchronous scheduling more visible.

Runtime Policy as a First-Class Design Issue. The fanout experiments also show that asynchronous structure alone is not enough. In AsyncMux, complexity

moves from explicit thread orchestration to worker-pool scheduling, queue management, and task admission. The throughput results suggest that this shift is worthwhile, but the p99 behavior makes clear that runtime policy strongly shapes the outcome. A more sophisticated scheduler could prioritize latency-sensitive work, adapt pool usage under bursty load, or treat foreground reads differently from background migration.

Software Engineering Benefits. A major motivation for AsyncMux is not only performance, but also clarity of implementation. Multi-tier operations are inherently compositional: split the request, launch per-block work, wait for the group, then assemble the result. Coroutine-based code matches that structure naturally and avoids explicit condition variables and cross-thread signaling in the main control flow. We do not quantify developer effort in this paper, but the implementation experience suggests that this is a meaningful advantage as the storage stack grows more complex.

Limitations and Future Work. Our prototype has several limitations. The runtime is intentionally simple, the evaluation uses one node and one policy configuration, and the experiments do not isolate the effect of background promotion as a separate variable. Future work includes improving runtime scheduling to control tail latency, studying hybrid designs that combine asynchronous and blocking techniques, and extending the approach to distributed multiplexed storage.

6 Conclusion

AsyncMux explores the role of execution model in multiplexed tiered storage. Rather than changing placement policy or storage layout, it changes how multi-tier operations are coordinated: block-level work is expressed as asynchronous tasks and composed at the logical request boundary.

The results show that this approach is especially valuable for read-heavy and fanout workloads, where AsyncMux improves throughput and scalability under concurrency. At the same time, the evaluation makes clear that asynchronous coordination is not a universal win: coordination-heavy workloads can still favor a blocking design, and tail latency remains a key challenge for the current runtime.

Overall, AsyncMux suggests that event-driven coordination is a promising foundation for multiplexed storage layers, particularly when the system must manage substantial fanout across heterogeneous tiers. Fully realizing that promise will require pairing the programming model with stronger runtime scheduling and latency control.

References

- [1] Rohan Kadekodi, Se Kwon Lee, Sanidhya Kashyap, Taesoo Kim, Aasheesh Kolli, and Vijay Chidambaram. Splitfs: Reducing software overhead in file systems for persistent memory. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2019.
- [2] Youngjin Kwon, Hannes Fingler, Tyler Hunt, Simon Peter, Emmett Witchel, and Thomas Anderson. Strata: A cross media file system. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2017.
- [3] Jian Xu and Steven Swanson. Nova: A log-structured file system for hybrid volatile/non-volatile main memories. In *USENIX Conference on File and Storage Technologies (FAST)*, 2016.
- [4] Jiyuan Zhang, Jongyul Kim, Chloe Alverti, Peizhe Liu, Weiwei Jia, and Tianyin Xu. Rethinking tiered storage: Talk to file systems, not device drivers. In *Workshop on Hot Topics in Operating Systems (HotOS)*, 2025.
- [5] Sheng Zheng, Mohammad Hoseinzadeh, and Steven Swanson. Ziggurat: A tiered file system for non-volatile main memories and disks. In *USENIX Conference on File and Storage Technologies (FAST)*, 2019.